Name : Vatsal Postiwala PRN No: 26070122509 Batch : B3(2025-2029)

ASSIGNMENT No: 26

TITLE: Write a Java program to create and execute multiple threads using the Runnable interface.

Problem Statement

Write a Java program to create and execute multiple threads using the Runnable interface.

Learning Objectives

To create multiple threads using the Runnable interface.

Theory

The Runnable interface provides an alternative approach to creating threads in Java. A class implementing Runnable defines the run() method that contains the code executed by the thread. The Runnable object is passed to a Thread object, which starts execution using the start() method. Unlike extending the Thread class, implementing Runnable allows a class to inherit from another superclass, making the design more flexible. Runnable separates task definition from thread execution, resulting in better code organization and reusability. It is widely used in multithreaded enterprise applications and thread pool implementations.

Code:

```
package E26;

import java.lang.*;

class SimpleTask implements Runnable {
   String threadName;

   public SimpleTask(String name) {
      threadName = name;
   }

   public void run() {
      for (int i = 1; i <= 3; i++) {
         System.out.println(threadName + " is running step " + i);
      }
   }
}

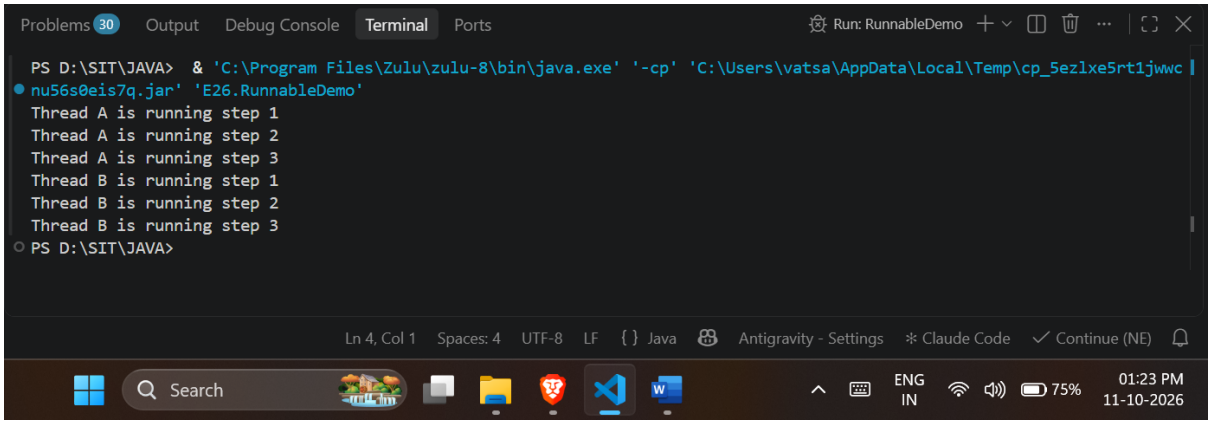
public class RunnableDemo {
```

```
    public static void main(String[] args) {
        SimpleTask task1 = new SimpleTask("Thread A");
        SimpleTask task2 = new SimpleTask("Thread B");

        Thread t1 = new Thread(task1);
        Thread t2 = new Thread(task2);

        t1.start();
        t2.start();
    }
}
```

Output:



Exercise

1. Create a program using Runnable to run two tasks simultaneously.

Code:

```
package E26;

import java.lang.*;

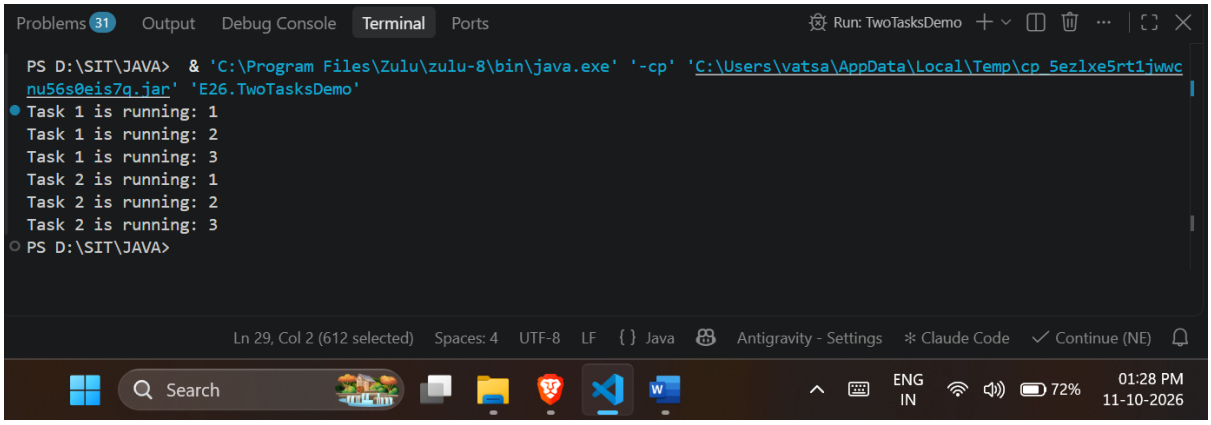
class TaskOne implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Task 1 is running: " + i);
        }
    }
}
```

```
class TaskTwo implements Runnable {
  public void run() {
    for (int i = 1; i <= 3; i++) {
      System.out.println("Task 2 is running: " + i);
    }
  }
}

public class TwoTasksDemo {
  public static void main(String[] args) {
    Thread t1 = new Thread(new TaskOne());
    Thread t2 = new Thread(new TaskTwo());

    t1.start();
    t2.start();
  }
}
```

Output:



2. Create a Java program using the **Runnable interface** where two tasks run simultaneously: one thread performs **deposit operations**, and another performs **withdrawal operations**, both displaying results concurrently.

Code:

```
package E26;

import java.lang.*;
```

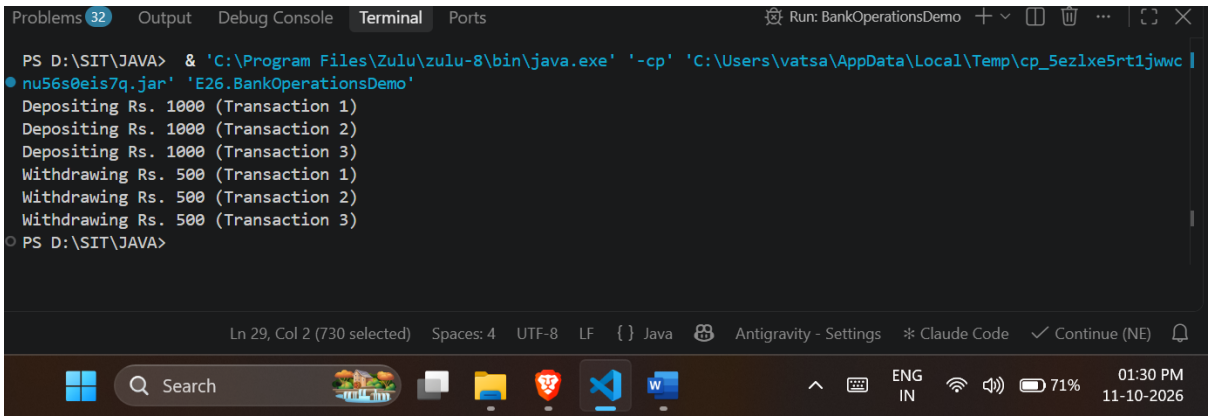
```
class DepositTask implements Runnable {
  public void run() {
    for (int i = 1; i <= 3; i++) {
      System.out.println("Depositing Rs. 1000 (Transaction " + i + ")");
    }
  }
}

class WithdrawalTask implements Runnable {
  public void run() {
    for (int i = 1; i <= 3; i++) {
      System.out.println("Withdrawing Rs. 500 (Transaction " + i + ")");
    }
  }
}

public class BankOperationsDemo {
  public static void main(String[] args) {
    Thread depositThread = new Thread(new DepositTask());
    Thread withdrawalThread = new Thread(new WithdrawalTask());

    depositThread.start();
    withdrawalThread.start();
  }
}
```

Output:



Conclusion:

I learned how to create and execute multiple threads simultaneously by implementing the Runnable interface and overriding the run() method. I also understood that this approach is more flexible than extending the Thread class because it allows a class to inherit from other superclasses while still supporting multithreading. Finally, I learned how to practically apply this concept to run independent tasks, such as bank deposits and withdrawals, concurrently by passing the Runnable objects to Thread instances and invoking the start() method.